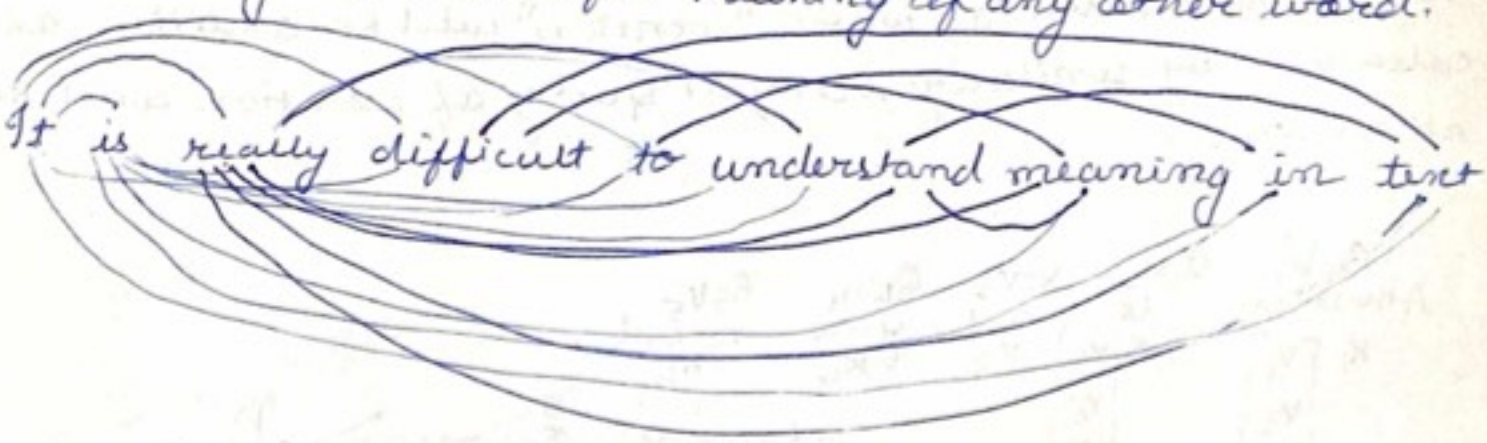
Attention mechanism

why it is so hard to understand text?

Imagine a large text with multiple pages. Any word in later part of pages can be related to initial text words.
This means any text can influence meaning of any other word.



If you think this looks complicated imagine what happens when you upload a document with 20 pages

we are not just trying to understand what part of text is related to other part but also in what way they are related

In attention mechanism we create these associations between words by using two vectors for each word Q (Query) and K(Key)
These two vectors will tell which words are related to which other word.

K	K	K	K	K
Attention	is	all	you	need
Q	Q	Q	Q	Q

Here Q and K values are different for each word.

for ex:- Attention is

$$\begin{array}{cccc} Q & K & Q & K \\ \begin{bmatrix} 0.4 \\ -0.2 \\ 0.3 \\ 0.25 \end{bmatrix} & \begin{bmatrix} 0.1 \\ 0.6 \\ -0.1 \\ -0.5 \end{bmatrix} & \begin{bmatrix} 0.2 \\ 0.3 \\ -0.2 \\ -0.4 \end{bmatrix} & \begin{bmatrix} 0.65 \\ 0.7 \\ -0.1 \\ 0.25 \end{bmatrix} \end{array}$$

The Q of Attention will check similarity with K vector of all other words. The more similar they are the more strongly related they are.
But we haven't yet got a way to indicate strength or volume of the similarity. Hence we add a third vector to each word V.
V will indicate how much two words are related.

Q Q Q Q Q
Attention is all you need
K V K V K V K V K V

The value of V for a word is determined by similarity between Query of the word and ~~value~~ Key of every other word.

For example value of word "attention" will be addition of values calculated by similarity between Query of attention and key of all other words.

Q_1V_1 Q_2V_2 Q_3V_3 Q_4V_4 Q_5V_5
Attention is all you need.
K_1 K_2 K_3 K_4 K_5

$$\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix} \quad \begin{bmatrix} v_5 \\ v_6 \\ v_7 \\ v_8 \end{bmatrix}$$

~~where $v_1 = \sum_{Q=0}^{5} \text{Similarity}(Q_1, K)$~~

where V_1 = Sum of similarity of Q_1 and $[K_2, K_3, K_4, K_5]$

capital $V_1 = v_1 + v_2 + v_3 + v_4$

~~Now Attention is more similar~~

- Query of attention is more similar to "need" and "is" than to other words. Hence Value calculated by similarity Q_1 and K_5 as well as Q_1 and K_2 i.e. V_1 and V_4 are stronger than V_2 and V_3.

Now instead of one value for each word we have a sum of all values determined by combination of Query and Key of all words.

This ~~sum of~~ new value will capture meaning or relation but also strength of that relation.

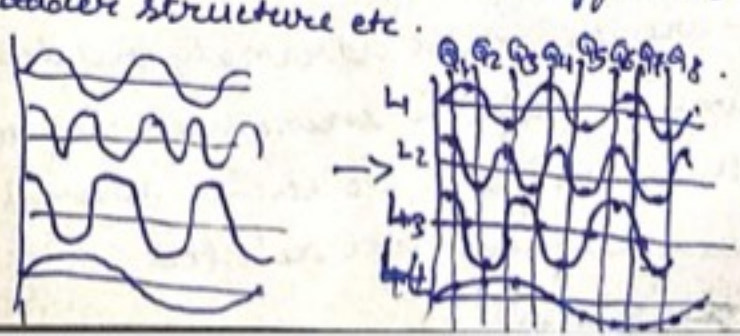
There is still one thing remaining.

The order of words are not considered while creating values.

Now we can do this by adding more dimensions to V before calculating similarity.

These more dimensional values we add to indicate how order influences meaning of word or value V is called positional ~~embeddings~~ encodings

Positional encodings are calculated by calculating sine transforms of different frequencies which capture different aspects of positions like local positions, broader structure etc.



$$\longrightarrow \begin{bmatrix} Q_{11}, Q_{12}, Q_{13}, Q_{14}, Q_{15}, Q_{16}, Q_{17}, Q_{18} \\ Q_{21}, Q_{22}, Q_{23}, Q_{24}, Q_{25}, Q_{26}, Q_{27}, Q_{28} \\ \cdots \\ \cdots \end{bmatrix}$$

Positional encodings are then combined with word embeddings matrix (Addition) to get final embeddings which ~~are~~ contain meaning and position relevance.

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}}) = \sin\left(\frac{t}{N^{\left(\frac{K}{d_{model}}\right)}}\right)$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}}) = \cos\left(\frac{t}{N^{\left(\frac{K}{d_{model}}\right)}}\right)$$

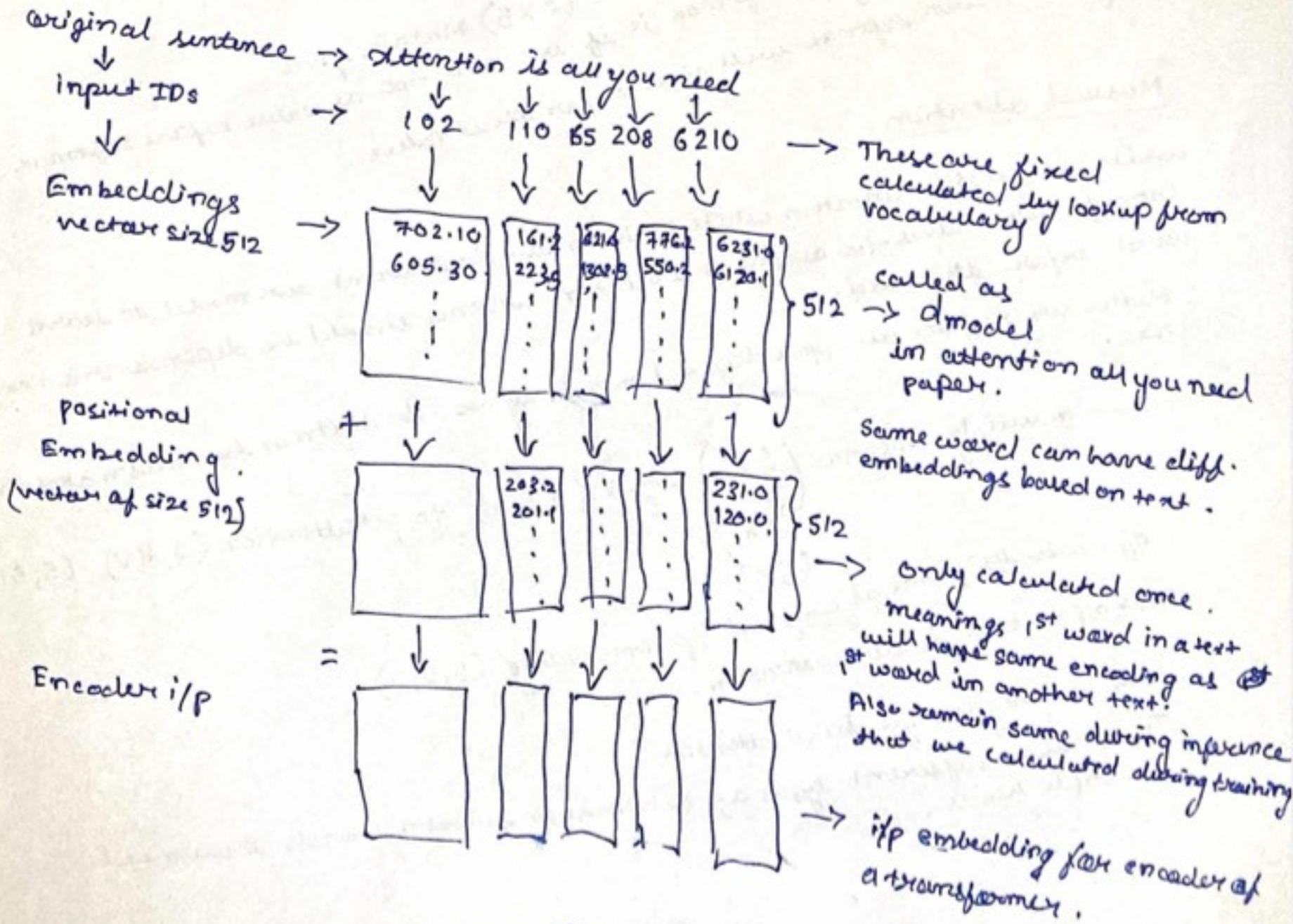
t = time duration

d_{model} = dimension of i/p embedding

Positional encodings must satisfy following properties

1. Unambiguous (position $\Leftrightarrow$ value)
   Each position should have unique value
2. Deterministic (based on underlying rule)
   Same sentence should always have same positional encodings
3. Allows to estimate distance between token's positions
4. Works with sequences longer than encountered before
   Trained on smaller sentences but can calculate longer sentences

Sine, cosine satisfy all 4 properties.



These i/p embeddings will contain :-

1) Meaning (Symantic) of the words in the sentence
2) Positional encoding of the words in the sentence
3) Relationship of every word in sentence with all other words in sentence.

Self Attention

Each word in sentence is related to all other words in sentence.

Lets say our sentence has 5 words, each word is represented by a vector of 512 dimensions. then attention $(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_K}}\right)V$

$$\text{Softmax}\left(\frac{\underset{(5,512)}{Q} \times \underset{(512,5)}{K^T}}{\underset{(5,5)}{\sqrt{512}}}\right) =$$

Attention	Attention	is	all	you	need	After Softmax
	-Some value	$-\infty$	$-\infty$	$-\infty$	$-\infty$	0
is	Some value	-	$-\infty$	$-\infty$	$-\infty$	1
all	-	-	-	$-\infty$	$-\infty$	0
you	-	-	-	-	$-\infty$	0
Need	-	-	-	-	-	0

(5×5) Matrix gives

one important property of softmax is if we add $-\infty$ as value before softmax for any value then softmax will give 0 for that value.

Masked attention

while calculating attention while training we dont want our model to learn from future values. i.e. output at a point in sentence should be dependent on only words before the word.

Hence we replace all upper diagonal matrix by $-\infty$. So softmax ~~becomes~~ makes them zero.

Next we multiply $\text{Softmax}\left(\frac{QK^T}{\sqrt{d_K}}\right)$ $(5,5)$ by V $(5,512)$ to get attention (Q, K, V) $(5, 512)$

o/p will be same as original i/p embedding $(5, 512)$

This o/p matrix is our attention.

This is only single head attention.

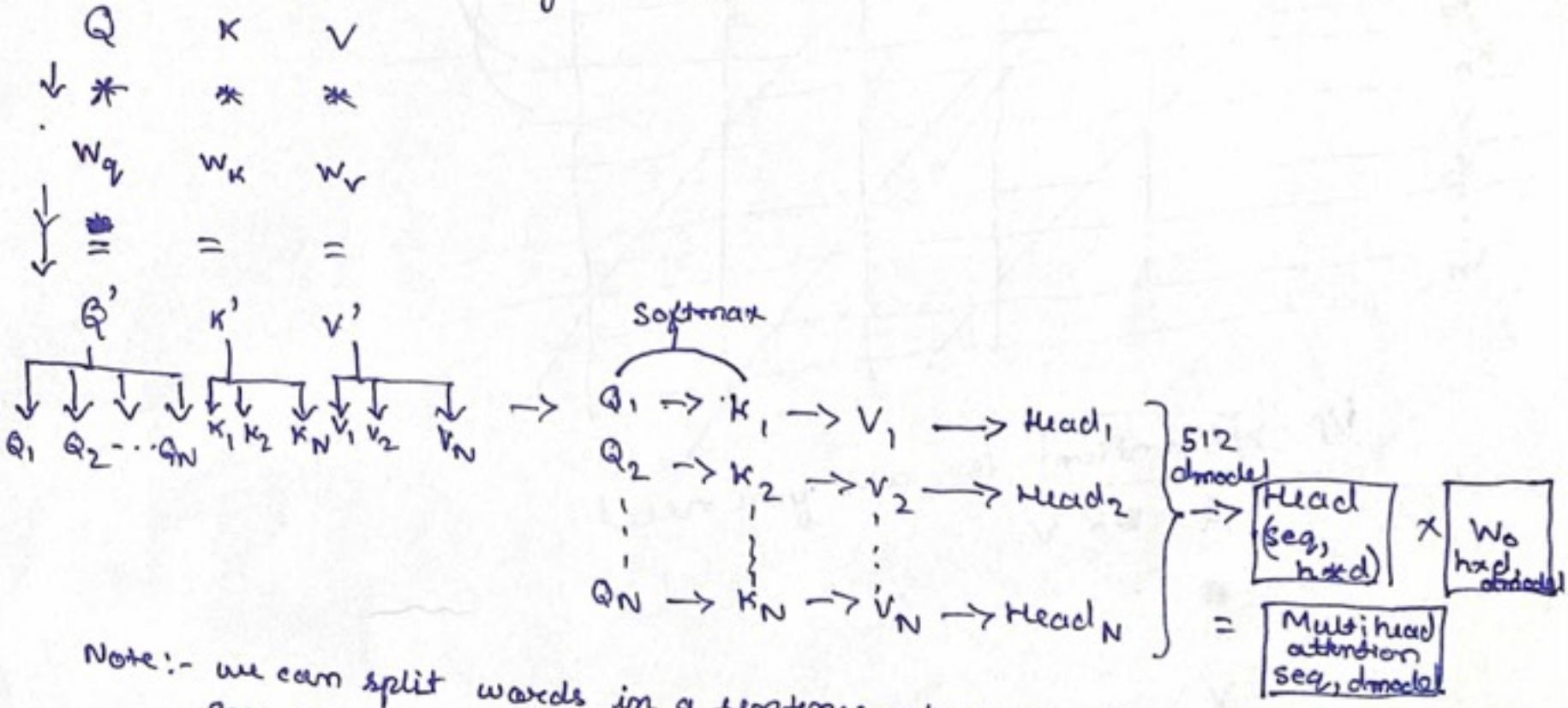
To calculate different types of relationship between words we need multiple heads.

Multi head attention

To calculate or understand meaning of words in different aspects, we split the i/p embeddings in multiple parts before calculating attention. we calculate attention for each part called as heads. These attentions or heads are then concatenated and Multiplied by weight matrix W to get Multihead attention.

example, lets consider a word in following sentence.

$\boxed{\text{Attention}}$ is all you need



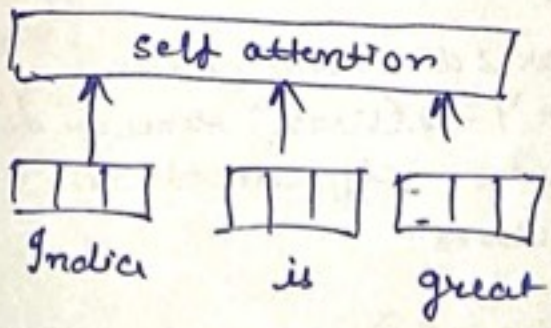
Note:- we can split words in a sentence along 512 dimensions into lets say 5 parts or we can split sentence in smaller parts. In transformer we always split words along 512 dimensions of vector length into smaller parts lets say 64x64 i.e. 8 heads.
So each head will see smaller part of each word embedding but will see the whole sentence.

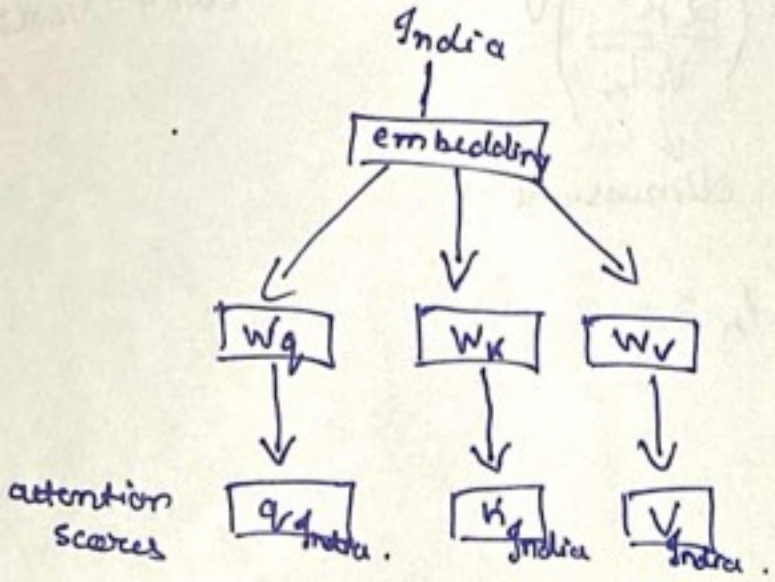
Cross Attention (Encoder-Decoder attention

- This attention focuses on different input sequences to find relationship between tokens in different sequences.

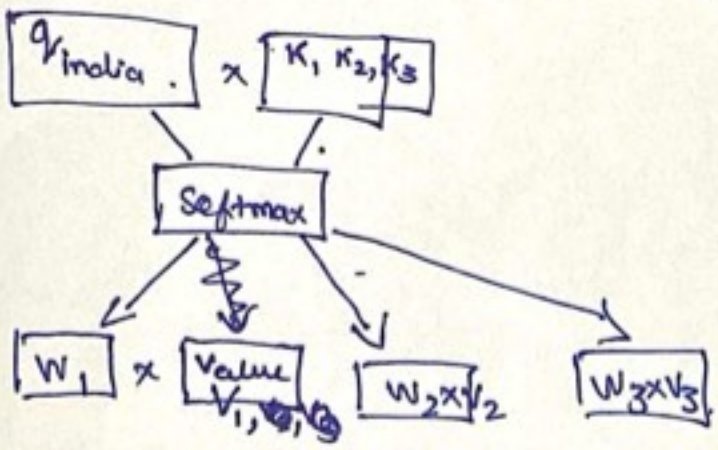
Also called as encoder-decoder attention.

Self Attention



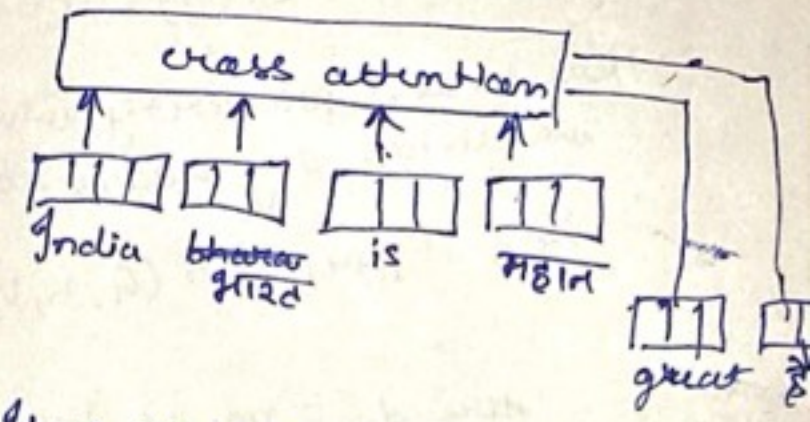


Now q of India is compared with Key K of 'is', 'great' and also of 'India' itself



Here V_1, V_2, V_3 are values produced by original Multiplication of values of each word multiplied with weights of q and K.

Cross Attention



If we are trying to translate.

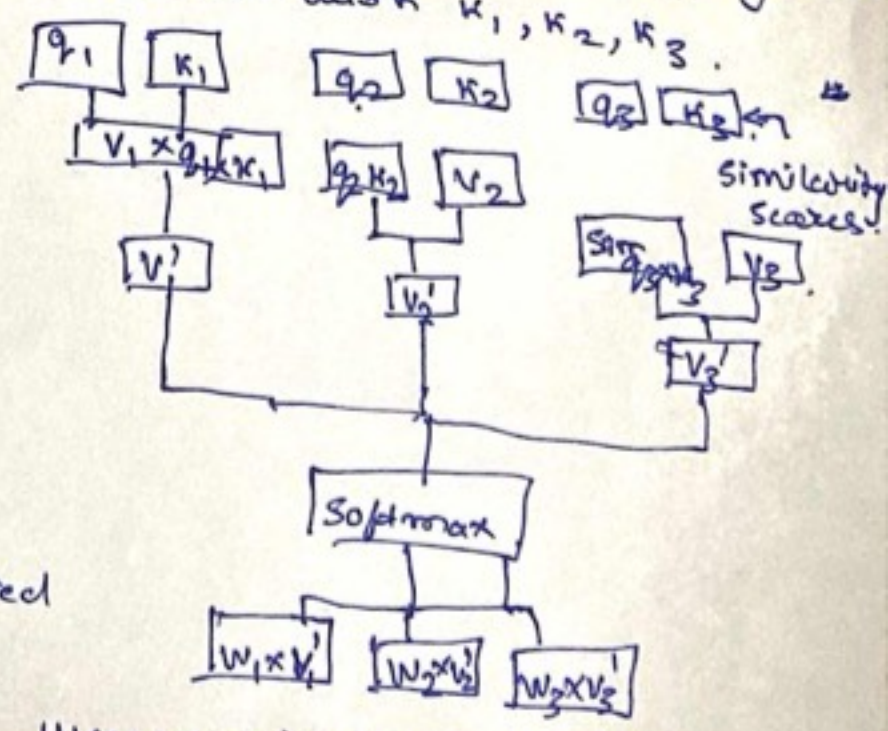
India is great. → भारत महान है

i/p o/p.

In cross attention we take query vector Q from o/p seq i.e. e.g. भारत and key, value vector pairs from i/p i.e. India is great

K_1, V_1 K_2, V_2 K_3, V_3.

Now q of India is compared i.e. similarity of q_1 i.e. 'India' query vector is calculated with K_1, K_2, K_3.



Usecase:- i/p image to o/p text.

Speech to text.

Scaled dot product

If a random variable x with variance $Var(x)$ and if we scale x by a scalar c such that $y = cx$ then variance y ~~is equal~~ scales by square of c i.e. c^2

i.e. if $y = cx$ and $x \rightarrow Var(x)$

then $Var(y) = c^2 Var(x)$

for 2 dimension vector. $Var\, y = 2(Var\, x)$ Hence we do $\frac{Var(y)}{\sqrt{2}} = \left(\frac{1}{\sqrt{2}}\right)^2 2V$

$= \frac{1}{2} \times 2\, Var$

$Var(y) = Var(x)$

Due to this property, when we calculate self attention, we divide by square root of dimensions.

i.e.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_K}}\right)V$$

here d_K = no. of dimensions in our i/p embedding. dimensions.

In original transformer paper $d_K = 512$